Background execution and power management

App lifecycle, background execution, and energy

Dinu-Ștefan Rusu MEC

What you should leave with

App states

Foreground, background, suspended and killed on Android and iOS, and what your process keeps.

Background scheduling

Doze, standby buckets, foreground services, WorkManager and BGTaskScheduler.

Background work in Flutter

The workmanager plugin, background isolates, push wakes, and the limits you cannot argue with.

Energy budgeting

Where the milliamp-hours go, the radio tail, batching, and how to measure the result.

1

App lifecycle

App states, suspension, and process termination

Contents

		Page
1	**App lifecycle**	3
2	Android background restrictions	9
3	iOS background execution	16
4	Background work in Flutter	22
5	Battery as a design constraint	29

Foreground, background, and not running

Foreground

On screen. A full CPU share, the network, and every permission the user granted.

Background

Off screen but still alive for a short while.

Not running

The process is gone. Your objects, timers, sockets and subscriptions went with it.

Neither platform allows an ordinary app to keep running because it would like to. Background execution is granted for a declared reason, for a limited time, and it can be withdrawn.

Persist state before the process is killed.

Android process importance

Rank	Process	What it means
1	Foreground	An activity on screen, or a foreground service.
2	Visible	Visible but not in focus, behind a dialog.
3	Service	Work the user cannot see, started by you.
4	Cached	Nothing running. Kept only while memory is free.

The low memory killer walks up from the bottom. A cached process just disappears.

Nothing warns a cached process. onPause and onStop are the last moments you are guaranteed. Save your state there, not later.

iOS app states

State	What runs
Not running	Nothing. Never launched, or terminated.
Inactive	Running, receiving no events: a call arrives.
Active	The normal foreground state.
Background	Code still runs, briefly, or for a declared mode.
Suspended	In memory, no code runs. Terminated without notice.

Suspended is where most apps spend the night, and Jetsam can terminate them there.

Suspended is not running. A timer you started and a socket you held are frozen until the user opens the app again.

Causes of process termination

Memory pressure

The most common cause. A large photo cache makes you an easy target.

Energy pressure

Repeated wakeups and held wakelocks push you down the standby buckets.

The user

A swipe out of the app switcher on iOS stops background delivery until the app is opened again.

Updates and reboots

Your process ends. Only work registered with the system survives a reboot.

Watchdog timeouts

Blocking the main thread while launching or backgrounding is a crash, not a slowdown.

Policy

A misdeclared foreground service can end the process on sight.

Lifecycle events in Flutter

```
@override
void didChangeAppLifecycleState(
    AppLifecycleState s) {
  if (s == AppLifecycleState.resumed) {
    sync.now();
  }
  if (s == AppLifecycleState.paused) {
    repo.flush();  // last safe moment
  }
}
```

Five states, one callback

`resumed`, `inactive`, `paused`, `hidden`, `detached`. Add the observer in `initState`, remove it in `dispose`.

Treat `paused` as the last line of your program.

Sync on resume and on regaining connectivity. A resume handler and a connectivity handler will do more syncing than any background scheduler.

2

Android background restrictions

Doze, buckets, foreground services, WorkManager, alarms

Contents

		Page
1	App lifecycle	3
2	**Android background restrictions**	9
3	iOS background execution	16
4	Background work in Flutter	22
5	Battery as a design constraint	29

Doze mode

Screen off, unplugged, still
The system enters Doze. Network access is cut for apps, alarms and jobs are deferred, and wakelocks are ignored.

Work is released in maintenance windows that grow further apart the longer the device stays idle.

What still gets through

High-priority messages from Firebase Cloud Messaging, exact alarms, and whatever a foreground service is doing.

”Every 15 minutes” means ”no sooner than 15 minutes”. Under Doze the gap between two runs of a periodic job can be hours.

App Standby and the standby buckets

Bucket	Who lands here	Effect
Active	Used right now	No restriction.
Working set	Used most days	Jobs and alarms deferred slightly.
Frequent	Used regularly	Deferred more, fewer priority messages.
Rare	Rarely opened	Jobs and network heavily limited.
Restricted	Heavy background use	Roughly one job window a day.

The bucket is per app and it moves. There is no API that sets your own.

Test on a phone that is not yours. Your development phone sits in the active bucket all day. A tester’s phone that opens the app once a week does not.

Foreground services

A visible notification

The user can see the service and stop it.

A declared type

Since Android 14 every one needs a type in the manifest with a matching permission: location, data sync, connected device, media.

Limited entry

You generally cannot start one while the app is in the background. Start it from a screen the user is on.

A foreground service is not a way to avoid the scheduler. Use one for work that is ongoing, user-initiated and user-visible: a recorded track, a live sensor session, an upload the user just asked for.

WorkManager

Deferrable and guaranteed

The work is persisted, so it survives process death, app updates and reboots. It runs when the constraints are met, not when you asked.

Periodic work has a floor of roughly fifteen minutes, and several apps may be batched into one wakeup.

What you declare

Constraints: network connected or unmetered, charging, battery not low, device idle.

Backoff: linear or exponential, driven by a retry result rather than an exception.

Uniqueness: a name and a policy, so a resumed app does not enqueue the job twice.

Return a retry, do not throw. A worker that fails on a flaky network should ask to be retried so the backoff policy applies.

Exact alarms

An alarm fires at a wall-clock time
It can wake the device out of Doze. That is a cost paid by the whole system, so the permission to schedule one is restricted.

Recent Android versions grant it to apps whose purpose is timing: an alarm clock, a calendar, a countdown. Anything else must ask the user.

⚠ Wrong reasons to use one

Polling a server. Refreshing a feed. Making a periodic sync feel more reliable. All three are WorkManager jobs.

Would the user notice a delay of ten minutes? If the answer is no, an exact alarm is the wrong tool and it will be rejected at review.

Battery optimization exemptions

What it does

Taking your app out of battery optimization lifts App Standby and most Doze limits on your jobs.

What it costs

Store policy allows the prompt only for a narrow set of app categories. Asking without one risks the listing.

Why it fails anyway

The user can revoke it, many vendors add their own killer on top, and it does not fix a design that wakes too often.

Do not rely on a battery exemption. If the app only works when it is exempt, the work is scheduled wrong. Fix the schedule instead, and test on a device from a vendor that ships extra restrictions.

3

iOS background execution

Background modes, background time, BGTaskScheduler, silent push

Contents

		Page
1	App lifecycle	3
2	Android background restrictions	9
3	**iOS background execution**	16
4	Background work in Flutter	22
5	Battery as a design constraint	29

Background modes

Fetch and processing
Short refreshes, and longer maintenance work.

Location
Continuous or significant-change updates.

Audio
Playback and recording off screen.

Bluetooth
Central or peripheral.

Remote notification
Woken for a silent push.

External accessory
A connected accessory.

A mode describes a feature the user can see. Declaring one you do not visibly use is the most common reason for a rejection.

Background time limit

Leaving the foreground starts a timer
You get roughly thirty seconds to finish what you were doing before the process is suspended. Then no code runs.

An app can request a short extension for one task it is finishing, must supply an expiration handler, and must end the task itself. Forgetting is a crash, not a leak.

What this means in Dart

An upload started in the paused handler may be cut off mid-request. Make the server side idempotent and record what you finished.

Do not start long work while going to the background. Write your queue to disk instead, and let the next launch or a scheduled task drain it.

BGTaskScheduler: refresh and processing

	App refresh	Processing
Purpose	Pull new content	Cleanup, migration, a large sync
Budget	Tens of seconds	Minutes, decided by the system
Condition	When you are likely to be opened	Can require power and network
When	Spread through the day	Usually overnight, on a charger

Since iOS 26 there is a third kind, a continued processing task: work the user starts in the foreground keeps running with visible progress they can cancel.

Nothing here is guaranteed. The system schedules you from how often the user opens the app. A rarely used app may not run for days.

Silent push notifications

A push with no user-visible content
The payload marks the message as content available and low priority. The system wakes the app briefly to fetch what changed.

It is throttled per app, dropped in low power mode, and not delivered at all after the user force quits the app.

⚠ Do not carry data in it

Treat a silent push as a hint that something changed. Fetch the actual change from your server, so a dropped message costs nothing.

Use a visible notification when the user must know. A visible alert is delivered reliably. A silent one is best effort, on both platforms.

Android and iOS compared

Need	Android	iOS
Deferrable sync	WorkManager	BGAppRefreshTask
Long maintenance	WorkManager, charging	BGProcessingTask
Ongoing visible work	Foreground service	A declared background mode
Wake on a server event	High-priority message	Silent push
Fire at a wall-clock time	Exact alarm, restricted	Local notification only

One abstraction in Dart can cover both shapes. The guarantees do not match.

The guarantees differ by platform. Android promises the work happens eventually. iOS only promises to offer the chance.

4

Background work in Flutter

Plugins, background isolates, push wakes, and limits

Contents

		Page
1	App lifecycle	3
2	Android background restrictions	9
3	iOS background execution	16
4	**Background work in Flutter**	22
5	Battery as a design constraint	29

The workmanager plugin

```dart
@pragma('vm:entry-point')
void callbackDispatcher() {
  Workmanager().executeTask((t, d) async {
    await Repo.syncOnce();   // opens the db
    return true;   // false asks a retry
  });
}
Workmanager().initialize(callbackDispatcher);
Workmanager().registerPeriodicTask('sync',
    'sync', frequency: Duration(minutes: 15));
```

One Dart entry point
The dispatcher is a top-level function, marked so the compiler keeps it in a release build.

On Android this becomes WorkManager. On iOS it maps onto background fetch and processing, with all of that platform's uncertainty.

Fifteen minutes is a floor, not a promise. Register once, keep the task idempotent, and let the system pick the moment.

Background isolates

No widgets, no context

There is no UI, no navigator and no BuildContext. Anything that touches the tree will throw.

No shared memory

Globals, singletons and your BLoC instances do not exist here. Open the database again and read state from disk.

Plugins need registering

Some plugins only work once the background engine registers them. Test in a release build, not only in debug.

Write the result down, then let the task end. The app reads it on the next launch. That is how Lab 4 proves the task ran while the app was closed.

flutter_background_service

A long-running isolate

On Android it can run as a foreground service with a notification, so it keeps a socket or a BLE connection alive while the screen is off.

On iOS there is no equivalent. The same code gets a few seconds now and then instead of a continuous session.

When to reach for it

Continuous device work the user can see: an MQTT session, a BLE stream, a recorded track. Not periodic sync.

Pick the tool by the shape of the work. Periodic and deferrable goes to workmanager. Continuous and visible goes to a foreground service.

Waking the app with a data message

```
@pragma('vm:entry-point')
Future<void> onBackground(
    RemoteMessage m) async {
  await Firebase.initializeApp();
  await Repo.open();
  await Repo.syncOnce();  // fetch a delta
}
// in main(), after initializeApp()
FirebaseMessaging
    .onBackgroundMessage(onBackground);
```

The most reliable wake there is
A data message with high priority can pull an Android device out of Doze. On iOS it becomes a silent push.

The handler runs in its own isolate, so Firebase and the database are opened again.

Wake, fetch a delta, write it down, finish. The window is short. Do not stream, and do not retry in a loop.

Geofencing and location wakes

Location is the strongest wake source
Both platforms can relaunch a terminated app when the device enters or leaves a region, or moves far enough.

The number of monitored regions is small and the crossing is detected with a delay, because the system uses cheap signals first.

⚠ The permission is the hard part

Background location needs a separate grant and a written justification at review. Ask only if the feature is why the user installed the app.

Do not use a geofence as a general-purpose timer. It is a location feature that happens to wake you. Users read it as tracking, and reviewers do too.

Background execution limits

Hold a socket open

A WebSocket or MQTT session does not survive suspension, nor Doze on Android.

Run on a precise schedule

A periodic task is a request. No API on either platform promises a run every minute.

Watch sensors freely

Continuous sensor or scan work off screen needs a declared reason, and is throttled.

Restart yourself

An app the user force quit stays stopped until the user opens it again.

Show UI

No dialogs and no navigation from a background isolate. A notification is the only channel.

Assume time passed

Read the clock and the stored cursor on wake. Never assume one interval went by.

5

Battery as a design constraint

Screen, silicon, and radio

Contents

		Page
1	App lifecycle	3
2	Android background restrictions	9
3	iOS background execution	16
4	Background work in Flutter	22
5	**Battery as a design constraint**	29

Power consumption by component

The screen

On an OLED panel a black pixel is unlit, so dark mode saves energy. On an LCD the backlight is on regardless.

CPU and GPU

Inefficient code raises load, load raises temperature, and the system throttles the chip.

The radio

The most expensive component per byte, and the one that keeps drawing power after your request has finished.

Race to idle. A modern chip is most efficient when it finishes quickly and sleeps. A poll loop or a busy animation costs more than the work it performs.

The radio tail

A cellular radio does not switch off when your response arrives. It stays in a high-power connected state for several seconds first, in case more data is coming. The network decides this, not you.

A transfer of 200 bytes can keep the radio awake for seconds.
An app that sends one small packet every few seconds never allows the radio to step down at all.

What to do about it

Batch. Collect analytics, logs and non-urgent writes and flush them together.

Defer. Let the scheduler run the work when the radio is already awake.

Prefetch on Wi-Fi and while charging. Replace polling with push. Send less.

Group your network calls into bursts. The fixed cost is the radio wake-up rather than the bytes, so ten small transfers cost far more than one batched transfer.

Wakelocks

A wakelock blocks sleep
It tells Android to keep the CPU running while you finish something. A leaked one keeps the device awake until the battery is flat.

You rarely need one. WorkManager holds one for the duration of your worker, and a foreground service keeps the process alive for you.

⚠ The classic leak

Acquire, then hit an exception before the release. Release in a finally block, and always set a timeout.

If you are writing wakelock code, check the scheduler first. Almost every case that seems to need one is a job that should have been handed to the system.

Energy measurement

```
# put the device into Doze, now
adb shell dumpsys deviceidle force-idle
adb shell dumpsys deviceidle unforce

# which bucket is my app in
adb shell am get-standby-bucket com.x.app

# the raw counters, and a report
adb shell dumpsys batterystats --reset
adb bugreport report.zip
```

Android: read the bug report
Battery Historian draws a timeline from it: wakelocks, jobs, alarms, radio and screen state. It is unmaintained now, so prefer the Power Profiler or Perfetto.

iOS: the energy gauge
The debug navigator rates energy impact live, and Instruments has an energy template. MetricKit reports the same categories from real users.

Measure one number for the project. The project asks for a claim you measured, with the method written down.

Background sync task design

- [ ] Writes go to the local database and an outbox in one transaction, so nothing depends on the sync running.
- [ ] Foreground triggers first: sync on resume, and on regaining connectivity.
- [ ] One scheduled task on top, with a network constraint and a retry result on failure.
- [ ] The task opens its own database, reads a stored cursor, never assumes how long it slept.
- [ ] The work is idempotent and the drain bounded: a task cut off halfway is safe to repeat.
- [ ] State is written to disk before the task returns, and the app reads it on the next launch.

This is the shape of Task III in Lab 4: a periodic task that writes a timestamp and the last reading, visible after the app was closed.

Summary

1. **Background execution is granted by the system.** Declare a reason, accept a budget, and expect to be killed.
2. **Foreground triggers do most of the syncing.** Sync on resume and on reconnect first.
3. **The radio dominates the battery drain.** Batch, defer, and prefer one wake to twenty small ones.

Further reading

Doze and App Standby · WorkManager · BackgroundTasks · pub.dev/packages/workmanager

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel